

UNIVERSITÀ DEGLI STUDI DI UDINE

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Corso di Laurea in Informatica

Progetto di Laboratorio di Basi di Dati

05/06/2026

Gioele Vuaran
167222@spes.uniud.it

Filippo Nassivera
165594@spes.uniud.it

Mattia Rossetto
169423@spes.uniud.it

Francesco Viciguerra
166896@spes.uniud.it

Anno Accademico 2025/2026

Indice

1. Analisi dei Requisiti	2
1.1. Glossario	2
1.2. Strutturazione dei Requisiti	2
1.2.1. Ambiguità	3
1.3. Operazioni	3
2. Progettazione Concettuale	5
2.1. Vincoli di Integrità	5
3. Progettazione Logica	7
3.1. Ristrutturazione del Modello E-R	7
3.1.1. Tavola dei Volumi	7
3.1.2. Analisi delle Ridondanze	7
3.1.2.1. Ridondanza 1: Attributo derivato “Numero di Film Prodotti”	7
3.1.2.2. Ridondanza 2: Attributo derivato “Numero di Film Recitati”	8
3.1.3. Eliminazione delle Generalizzazioni	9
3.1.4. Traduzione degli Attributi Multivalore	9
3.1.5. Eliminazione della Relazione Quaternaria	10
3.1.6. Aggiunta di Chiavi Primarie Surrogate	10
3.1.7. Schema E-R Ristrutturato	10
3.1.8. Scelta degli Identificatori Primari	10
3.2. Traduzione nello Schema Relazionale	11
3.2.1. Vincoli di Integrità	12
4. Progettazione Fisica	14
5. Implementazione	18
5.0.1. Docker	18
5.1. Creazione della Base di Dati	18
5.2. Creazione delle Tabelle	19
5.3. Implementazione dei Trigger	19
5.3.1. Trigger 1: Una persona deve essere o un attore, o un regista, o entrambi	19
5.3.2. Trigger 2: Ci può essere al massimo un noleggio attivo	20
5.3.3. Trigger 3: Attributo derivato “Numero di Film Prodotti”	20
5.3.4. Trigger 4: Intervalli di noleggio non sovrapposti	21
5.4. Popolamento della Base di Dati	21
5.5. Interrogazioni ed Operazioni	22
5.5.1. Operazione 1	22
5.5.2. Operazione 2	23
5.5.3. Operazione 3	23
5.5.4. Operazione 4	24
5.5.5. Operazione 5	24
5.5.6. Operazione 6	25
5.5.7. Operazione 7	25
5.5.8. Operazione 8	26
5.5.9. Operazione 9	26
6. Conclusioni	28

1. Analisi dei Requisiti

Si vuole progettare una base di dati per la gestione di informazioni sull'industria cinematografica. La consegna segue dall'esercizio 3 del compito di basi di dati del 22 giugno 2009, con alcune aggiunte.

1.1. Glossario

È stato prodotto un glossario dei termini che compaiono nel testo, insieme ad eventuali sinonimi e collegamenti ad altri termini individuati.

Termine	Descrizione	Sinonimi	Collegamenti
Film			Attori, Azienda Produttrice, Registi, Frase Significative
Attore	Recita in uno o più film	Autore	Film
Regista	Dirige almeno un film, e può recitare in uno o più film		Film
Copia fisica di un Film*			Film, Cliente
Azienda Produttrice	Produce uno o più film		Film
Cliente*	Noleggia una o più copie fisiche di film	Cliente Registrato	Film
Ruolo	Svolto da un attore in un film, indica il personaggio interpretato (<i>es. Cenerentola</i>)		Attore, Film

Tabella 1: Glossario (* aggiunta alla consegna)

1.2. Strutturazione dei Requisiti

Si supponga di aver collezionato, dalla originale consegna, il seguente insieme di requisiti per la progettazione di una base di dati relazionale riguardante la gestione di informazioni sull'industria cinematografica.

Frasi relative a film
• Ogni film sia identificato univocamente dal suo titolo e dall'anno di produzione (assumiamo che in uno stesso anno non possano venir prodotti due o più film con lo stesso titolo, ma ammettiamo che film con lo stesso titolo possano essere prodotti in anni diversi, come accade nel caso dei remake). • Ogni film abbia una durata espressa in minuti, un'unica azienda produttrice e sia classificato come appartenente ad uno o più generi (commedia, thriller, film dell'orrore, fantasy, ..) • Ogni film abbia uno o più registi e zero, uno o più autori che vi recitano. • Ogni film sia caratterizzato da una breve descrizione della trama. • Ogni film abbia zero o più frasi significative, ciascuna pronunciata da uno degli attori che recitano nel film (assumiamo che alcuni attori possano pronunciare più frasi significative, altri una sola frase significativa, altri ancora nessuna).

Frasi relative a attori
• Gli attori siano identificati univocamente dal nome e dalla data di nascita (assumiamo che non vi siano attori con lo stesso nome nati lo stesso giorno). • Ogni attore compaia in almeno un film. • Ogni attore svolga uno o più ruoli in ogni film nel quale recita. • Si vuole registrare il numero di film in cui recita ogni attore.
Frasi relative a registi
• I registi siano identificati univocamente dal nome e dalla data di nascita (assumiamo che non vi siano registi con lo stesso nome nati lo stesso giorno). • Ogni regista diriga almeno un film. • Un regista possa anche recitare in uno o più film, inclusi film da lui/lei diretti.
Frasi relative a aziende produttrici
• Le aziende produttrici siano identificate dal loro nome e abbiano un unico recapito. • Ogni azienda produttrice produca uno o più film.

Si supponga di aver collezionato anche gli ulteriori requisiti, in aggiunta alla consegna originale.

Frasi relative a videonoleggio
Si vuole gestire il videonoleggio di film da parte dei clienti registrati. • Un cliente può noleggiare una o più copie fisiche di un film per un certo periodo di tempo limitato. Se una copia fisica risulta prestata, non può essere rinoleggiata prima che essa venga restituita. • Si vuole ricordare lo storico dei prestiti dei film da parte dei clienti.

1.2.1. Ambiguità

La consegna risulta estremamente chiara, avendo quindi solo un minimo numero di ambiguità.

È necessaria tuttavia una precisazione sul significato di *Ruolo*: come ruolo si intende il personaggio interpretato da un determinato attore nel film. Notiamo anche che tale personaggio viene considerato come specifico al film (ad esempio il ruolo di *Cenerentola* di un film risulterà differente dello stesso ruolo nel suo remake).

1.3. Operazioni

Le Operazioni 1-4 sono state definite per poter preparare delle interrogazioni, sviluppate in Sezione 5.5, mentre le Operazioni 5, 6, 7 e 8 con le relative frequenze per l'analisi dei costi e delle ridondanze, sviluppate in Sezione 3.1.2.

- *Operazione 1 (interrogazione)*: Ottieni il numero medio di attori che hanno partecipato in film di uno specifico genere.
- *Operazione 2 (interrogazione)*: Ottieni le coppie di clienti registrati che hanno visto gli stessi film.
- *Operazione 3 (interrogazione)*: Ottieni il regista che ha diretto il numero massimo di film.
- *Operazione 4 (interrogazione)*: Ottieni tutti gli attori che hanno recitato solo a film della stessa casa produttrice.

- *Operazione 5 (inserimento)*: Inserimento di nuovo film prodotto da una data casa produttrice. Frequenza: 57 inserimenti al giorno¹
- *Operazione 6 (interrogazione)*: Calcola il numero di film prodotti da una data casa produttrice. Frequenza: 50 richieste al giorno²
- *Operazione 7 (inserimento)*: Inserimento di una recitazione, con relativo ruolo e frase significativa. Frequenza: 570 inserimenti al giorno³.
- *Operazione 8 (interrogazione)*: Calcola il numero di film in cui un attore ha recitato. Frequenza: 100 richieste al giorno.
- *Operazione 9 (rimozione)*: Rimozione di un film.

¹Dal sito web IMDB, risulta che nell'anno 2024 sono stati rilasciati 20844 film, ovvero circa 57 film al giorno.

²Valore ipotetico - media rispetto a tutte le case produttrici.

³Circa 10 volte il numero di Film

2. Progettazione Concettuale

Dall'analisi del testo e dei requisiti è stato prodotto lo schema Entità-Relazioni di Figura 1.

La strategia di progettazione scelta è stata la strategia *inside out*: lo schema è stato quindi definito partendo dall'entità *Film* e, poco a poco, è stato allargato comprendendo il resto dei concetti identificati. È risultato utile per questo il Glossario, dal quale si è dedotto come *Film* fosse l'entità principale da cui tutte le altre dipendessero.

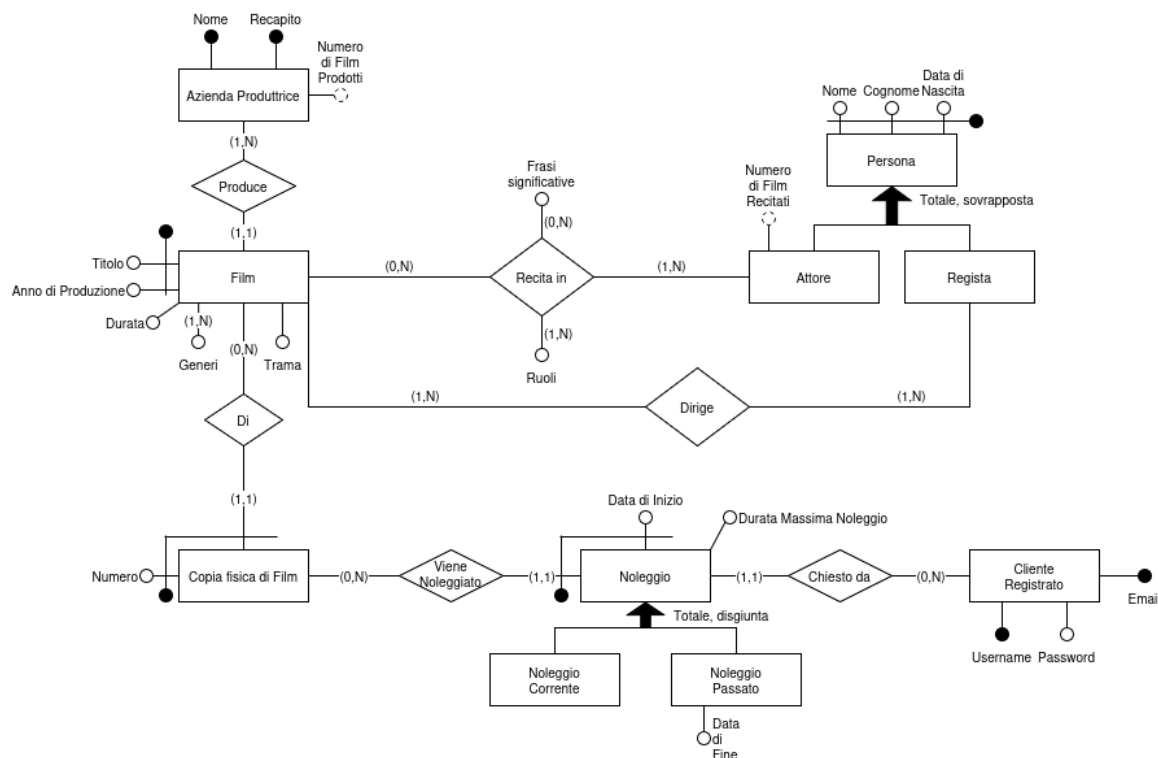


Figura 1: Schema Entità-Relazioni

Seguono alcune osservazioni sullo schema E-R:

- Per *Copia fisica di Film*, entità debole di *Film*, si è usato il pattern di progettazione *istanza di*: una copia fisica ha un numero che la identifica tra tutte le copie fisiche di un determinato film.
- In *Azienda Produttrice* viene mantenuto il numero di film da essa prodotti come un attributo derivato (derivato dalla relazione *Produce*).
- In *Attore* viene mantenuto il numero di film in cui ha recitato come un attributo derivato (derivato dalla relazione *Recita in*).
- Per i noleggi si è usato il pattern per la storicizzazione, quindi specializzando il *Noleggio* in *Noleggio Corrente* e *Noleggio Passato*; quest'ultimo possiede l'attributo *Data di Fine*.
- In *Cliente Registrato* sono presenti più chiavi candidate.
- In *Azienda Produttrice* sono presenti più chiavi candidate.

2.1. Vincoli di Integrità

I seguenti vincoli di integrità devono essere aggiunti al precedente schema per garantire il significato atteso della base di dati:

- Una Copia fisica di un Film noleggiata non può essere rinoleggiata prima che venga restituita (questo implica che gli intervalli temporali generati dalle date di inizio e fine noleggio non si sovrappongano).

- La Data di Inizio del Noleggio deve essere antecedente alla Data di Fine Noleggio.

Notiamo:

- Il ciclo '*Regista - Dirige - Film - Recita in - Attore*' non risulta problematico, in quanto uno specifico attore può recitare in un film che dirige.
- La durata massima del noleggio può essere maggiore della differenza tra la data di fine noleggio e la data di inizio noleggio, nel caso in cui la restituzione della copia fisica del film avvenga in ritardo.

3. Progettazione Logica

3.1. Ristrutturazione del Modello E-R

3.1.1. Tavola dei Volumi

Concetto	Tipo	Volume
Film	Entità	730000 ⁴
Produce	Relazione	730000
AziendaProduttrice	Entità	10000
Recitazione	Relazione	7300000 ⁵
Attore	Entità	2920000 ⁶

3.1.2. Analisi delle Ridondanze

Nello schema E-R sono presenti due ridondanze: l'attributo derivato *Numero di Film Prodotti* in *Azienda Produttrice* e l'attributo derivato *Numero di Film Recitati* in *Attore*. Nelle seguenti sezioni viene fatta un'analisi delle due ridondanze per decidere se mantenerle o eliminarle. Si assume un costo di un accesso in lettura di 1 unità di costo, e in scrittura di 3 unità di costo.

3.1.2.1. Ridondanza 1: Attributo derivato “Numero di Film Prodotti”

Presenza di ridondanza			
Operazione 5			
Concetto	Costrutto	Accessi	Tipo
Film	Entità	1	S
Azienda Produttrice	Entità	1	L
Azienda Produttrice	Entità	1	S
Produce	Relazione	1	S
$\begin{aligned}\text{Totale} &= (1 * \text{CostoLettura} + 3 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (1 * 1 + 3 * 3) * 57 \\ &= 570 \text{ unità di costo al giorno}\end{aligned}$			
Operazione 6			
Concetto	Costrutto	Accessi	Tipo
Azienda Produttrice	Entità	1	L
$\begin{aligned}\text{Totale} &= (1 * \text{CostoLettura} + 0 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (1 * 1 + 0 * 3) * 50 \\ &= 50 \text{ unità di costo al giorno}\end{aligned}$			
Totale = 570 + 50 = 620 unità di costo al giorno			

⁴Il sito web IMDB contiene 731089 film alla data di scrittura.

⁵Circa 10 volte il numero di Film

⁶Circa il 40% delle Recitazioni

Assenza di ridondanza			
Operazione 5			
Concetto	Costrutto	Accessi	Tipo
Film	Entità	1	S
Produce	Relazione	1	S
$\begin{aligned} \text{Totale} &= (0 * \text{CostoLettura} + 2 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (0 * 1 + 2 * 3) * 57 \\ &= 342 \text{ unità di costo al giorno} \end{aligned}$			
Operazione 6			
Concetto	Costrutto	Accessi	Tipo
Produce	Relazione	73 ⁷	L
$\begin{aligned} \text{Totale} &= (73 * \text{CostoLettura} + 0 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (73 * 1 + 0 * 3) * 50 \\ &= 3650 \text{ unità di costo al giorno} \end{aligned}$			
Totale = 342 + 3650 = 3992 unità di costo al giorno			

Dato che il costo in presenza di ridondanza, 620 unità al giorno, è minore del costo in assenza di ridondanza, 3992 unità al giorno, si sceglie di **mantenere** la ridondanza.

3.1.2.2. Ridondanza 2: Attributo derivato “Numero di Film Recitati”

Presenza di ridondanza			
Operazione 7			
Concetto	Costrutto	Accessi	Tipo
Recita in	Relazione	1	S
Attore	Entità	1	S
Attore	Entità	1	L
$\begin{aligned} \text{Totale} &= (1 * \text{CostoLettura} + 2 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (1 * 1 + 2 * 3) * 570 \\ &= 3990 \text{ unità di costo al giorno} \end{aligned}$			
Operazione 8			
Concetto	Costrutto	Accessi	Tipo
Attore	Entità	1	L
$\begin{aligned} \text{Totale} &= (1 * \text{CostoLettura} + 0 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (1 * 1 + 0 * 3) * 100 \\ &= 100 \text{ unità di costo al giorno} \end{aligned}$			
Totale = 3990 + 100 = 4090 unità di costo al giorno			

⁷Numero medio di Film prodotti per Azienda Produttrice = Volume(Produce) / Volume(Azienda)

Assenza di ridondanza			
Operazione 7			
Concetto	Costrutto	Accessi	Tipo
Recita in	Relazione	1	S
$\begin{aligned} \text{Totale} &= (0 * \text{CostoLettura} + 1 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (0 * 1 + 1 * 3) * 570 \\ &= 1710 \text{ unità di costo al giorno} \end{aligned}$			
Operazione 8			
Concetto	Costrutto	Accessi	Tipo
Recita in	Relazione	3 ^s	L
$\begin{aligned} \text{Totale} &= (3 * \text{CostoLettura} + 0 * \text{CostoScrittura}) * \text{FrequenzaOperazione} \\ &= (3 * 1 + 0 * 3) * 100 \\ &= 300 \text{ unità di costo al giorno} \end{aligned}$			
Totale = 1710 + 300 = 2010 unità di costo al giorno			

Dato che il costo in presenza di ridondanza, 4090 unità al giorno, è maggiore del costo in assenza di ridondanza, 2010 unità al giorno, si sceglie di **eliminare** la ridondanza.

3.1.3. Eliminazione delle Generalizzazioni

Nello schema E-R sono presenti due generalizzazioni:

- **Generalizzazione *Persona*:** Siccome la generalizzazione è totale e sovrapposta, usiamo il metodo ibrido per la rimozione della generalizzazione, in cui si pone ogni entità figlia in relazione con l'entità genitore. Le entità figlie diventano quindi entità deboli. Risulta necessario aggiungere un vincolo di integrità per fare in modo che *Persona* si specializzi obbligatoriamente in *Attore*, *Resista* o entrambi.
- **Generalizzazione *Noleggio*:** Siccome la generalizzazione è totale e disgiunta, applichiamo la tecnica della rimozione dei figli, spostando tutti gli attributi nella classe genitore. Per discriminare tra *Noleggio Corrente* e *Noleggio Passato* utilizziamo l'attributo "Data di fine", che ora diventa opzionale, come discriminante: se è presente, il noleggio è da considerarsi passato, altrimenti è corrente.

3.1.4. Traduzione degli Attributi Multivalore

Si considerino gli attributi *Frase significative* e *Ruoli* della relazione *Recita in*:

- ***Frase Significative*:** viene aggiunta un'entità debole in relazione (1,1) (nel lato *Frase Significativa*) con *Recita in*.
- ***Ruolo*:** aggiungiamo un'entità debole in relazione (1,1) (nel lato *Ruolo*) con *Recita in*

Si consideri l'attributo multivalore ***Generi*** in *Film*: viene aggiunta un'entità (non debole) in relazione (1, N) con *Film*.

⁸Volume(Recitazione) / Volume(Attore) = 2.3 arrotondato a 3

3.1.5. Eliminazione della Relazione Quaternaria

Siccome la relazione *Recita in*, a seguito della traduzione degli attributi multivalore, risulta quaternaria, è stata reificata in una nuova entità **Recitazione**, debole rispetto a *Film* e *Attore*.

3.1.6. Aggiunta di Chiavi Primarie Surrogate

Dal momento che l'entità *Frase Significativa* è identificata da una complessa chiave composta contenente anche una stringa possibilmente molto lunga (la frase stessa), è allora giustificabile usare una chiave surrogata per identificarla, sebbene non sia teoricamente necessario.

3.1.7. Schema E-R Ristrutturato

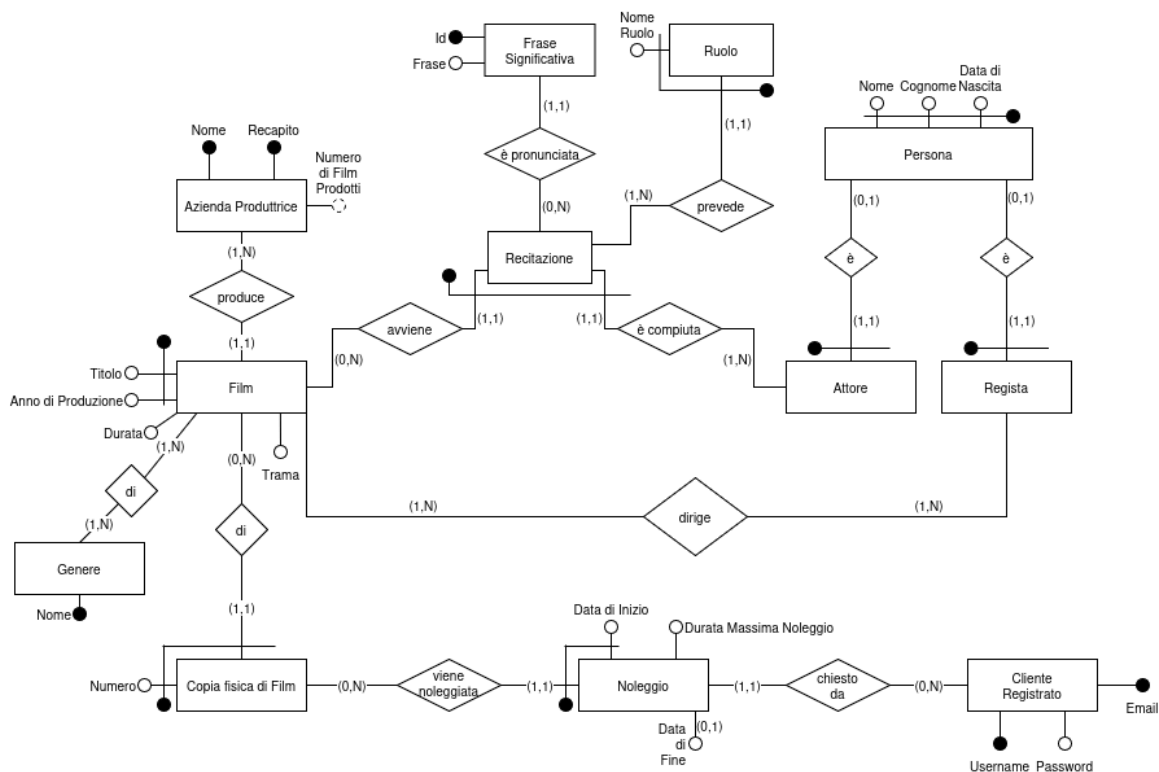


Figura 2: Schema Entità-Relazioni Ristrutturato

Vincoli di integrità:

- Una copia fisica di un film noleggiata non può essere rinoleggiata prima che venga restituita (questo implica che gli intervalli temporali generati dalle date di inizio e fine noleggio non si sovrappongano).
- La data di inizio del noleggio deve essere antecedente alla data di fine noleggio.
- Una *Persona* deve essere o un *Attore*, o un *Regista*, o entrambi.

3.1.8. Scelta degli Identificatori Primari

Si considerino le seguenti entità con più chiavi primarie candidate:

- **Cliente Registrato:** vi sono due chiavi candidate: *username* e *email*. Viene scelto **username**.
- **Azienda Produttrice:** abbiamo due chiavi candidate: *nome* e *recapito*. Viene scelto **nome**.

3.2. Traduzione nello Schema Relazionale

Segue la traduzione dello schema E-R ristrutturato allo schema relazionale:

AziendaProduttrice (Nome, Recapito, NumeroDiFilmProdotti)

VNN: {Recapito, NumeroDiFilmProdotti}

UNIQUE: {Recapito}

Film (Titolo, AnnoDiProduzione, Durata, Trama, AziendaProduttrice)

FK: {AziendaProduttrice} → {AziendaProduttrice.Nome}

VNN: {Durata, Trama, AziendaProduttrice}

Genere (Nome)

GenereDelFilm (TitoloFilm, AnnoDiProduzioneFilm, NomeGenere)

FK: {TitoloFilm, AnnoDiProduzioneFilm} → {Film.Titolo, Film.AnnoDiProduzione}

FK: {NomeGenere} → {Genere.Nome}

CopiaFisicaDiFilm (Numero, TitoloFilm, AnnoFilm)

FK: {TitoloFilm, AnnoFilm} → {Film.Titolo, Film.AnnoDiProduzione}

Noleggio (DataDiInizio, NumeroCopia, TitoloFilm, AnnoFilm, EmailCliente, DurataMassimaNoleggio, DataDiFine)

FK: {NumeroCopia, TitoloFilm, AnnoFilm} → {CopiaFisicaDiFilm.Numero, CopiaFisicaDiFilm.TitoloFilm, CopiaFisicaDiFilm.AnnoFilm}

FK: {EmailCliente} → {ClienteRegistrato.Email}

VNN: {EmailCliente, DurataMassimaNoleggio}

ClienteRegistrato (Email, Username, Password)

UNIQUE: {Username}

VNN: {Password, Username}

Persona (Nome, Cognome, DataDiNascita)

Attore (Nome, Cognome, DataDiNascita)

FK: {Nome, Cognome, DataDiNascita} → {Persona.Nome, Persona.Cognome, Persona.DataDiNascita}

Regista (Nome, Cognome, DataDiNascita)

FK: {Nome, Cognome, DataDiNascita} → {Persona.Nome, Persona.Cognome, Persona.DataDiNascita}

RegistaDelFilm (TitoloFilm, AnnoDiProduzioneFilm, NomeRegista, CognomeRegista, DataDiNascitaRegista)

FK: {TitoloFilm, AnnoDiProduzioneFilm, NomeRegista, CognomeRegista, DataDiNascitaRegista} → {Film.Titolo, Film.AnnoDiProduzione, Regista.Nome, Regista.Cognome, Regista.AnnoDiNascita}

Ruolo (NomeRuolo, TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)

FK: {TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore} → {Recitazione.TitoloFilm, Recitazione.AnnoFilm, Recitazione.NomeAttore, Recitazione.CognomeAttore, Recitazione.DataNascitaAttore}

FraseSignificativa (ID, Frase, TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)

FK: {TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore} → {Recitazione.TitoloFilm, Recitazione.AnnoFilm, Recitazione.NomeAttore, Recitazione.CognomeAttore, Recitazione.DataNascitaAttore}

VNN: {Frase, TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore}

Recitazione (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore, DataNascitaAttore)

FK: {TitoloFilm, AnnoFilm} → {Film.Titolo, Film.AnnoDiProduzione}

FK: {NomeAttore, CognomeAttore, DataNascitaAttore} → {Attore.Nome, Attore.Cognome, Attore.DataDiNascita}

3.2.1. Vincoli di Integrità

La traduzione dallo schema concettuale allo schema logico (schema relazionale) rende necessaria l'aggiunta dei seguenti vincoli di integrità causati dalla perdita di espressività del modello relazionale, che vanno a integrare i vincoli già evidenziati inizialmente.

I **vincoli intra-relazionali**, escludendo i vincoli di chiave primaria, di unicità, e di *NOT NULL* che sono stati già individuati in Sezione 3.2, sono i seguenti:

- *Noleggio*: $DataDiFine > DataDiInizio$ if $DataDiFine$ IS NOT NULL
- *Film*: $Durata > 0$

I **vincoli inter-relazionali**, escludendo i vincoli di chiave esterna che sono stati già individuati in Sezione 3.2, sono i seguenti. Per ogni vincolo sono riportate le azioni che potrebbero violare l'integrità della base di dati:

- Un'azienda produttrice deve aver prodotto almeno un film.
 - Inserimento in *Azienda Produttrice*
 - Cancellazione in *Film*
 - Modifica dell'attributo *AziendaProduttrice* di *Film*
- Un genere deve essere associato ad almeno un film
 - Inserimento in *Genere*
 - Cancellazione in *Film*
 - Modifica dell'attributo *Genere* nella relazione *GenereDelFilm*
 - Cancellazione dell'attributo *Genere* nella relazione *GenereDelFilm*
- Un film deve essere associato ad almeno un genere
 - Inserimento in *Film*
 - Modifica di *Genere* nella relazione *GenereDelFilm*
 - Cancellazione di *Genere* nella relazione *GenereDelFilm*
- Un Film deve avere almeno un Regista
 - Inserimento in *Film*
 - Cancellazione in *Regista*
 - Modifica di *RegistaDelFilm*
 - Cancellazione di *RegistaDelFilm*
- Una Recitazione deve prevedere almeno un Ruolo
 - Cancellazione in *Ruolo*
 - Inserimento in *Recitazione*
- Un Attore deve recitare in almeno un Film
 - Inserimento in *Attore*
 - Modifica in *Recitazione*
 - Cancellazione in *Recitazione*
- Un Regista deve dirigere almeno un Film
 - Inserimento in *Regista*
 - Cancellazione in *Film*
 - Modifica in *RegistaDelFilm*
 - Cancellazione di *RegistaDelFilm*
- Una persona deve essere o un attore, o un regista, o entrambi:

- Inserimento di *Persona*
- Cancellazione di *Attore*
- Cancellazione di *Regista*
- Ci può essere al massimo un noleggio attivo
 - Inserimento in *Noleggio*
 - Modifica in *Noleggio*
- Gli intervalli generati dalle date di inizio e di fine noleggio, per ogni copia fisica di un film, non devono sovrapporsi
 - Inserimento in *Noleggio*
 - Modifica in *Noleggio*
- Attributo derivato *Numero di Film Prodotti* in *Azienda Produttrice*
 - Inserimento in *Film*
 - Modifica di *AziendaProduttrice* nella relazione *Film*
 - Cancellazione in *Film*

Notiamo che, per gli quanto riguardano gli intervalli generati dalle date relative ai noleggi, si ipotizza di lavorare con intervalli chiusi. Segue, quindi, che se un noleggio termina il giorno 10, un nuovo noleggio per la stessa copia fisica di un film può iniziare a partire dal giorno 11.

4. Progettazione Fisica

In questa sezione viene riportato il codice DDL (Data Definition Language) SQL per la creazione delle tabelle definite in Sezione 3.2.

I vincoli di integrità *intra-relazionali* sono stati implementati a livello di tabella tramite l'uso di clausole CHECK.

Per quanto riguarda i vincoli *inter-relazionali*:

- **Cancellazioni:** per la gestione delle chiavi esterne in fase di cancellazione, si è optato per l'uso di clausole `ON DELETE NO ACTION` nella maggior parte delle relazioni; sebbene l'utilizzo di `ON DELETE CASCADE` mantenga comunque l'integrità della base di dati, si è preferito rifiutare le operazioni di rimozione piuttosto che propagare le cancellazioni a cascata, cosa che comporterebbe la rimozione di grandi parti della base di dati (si immagini per esempio la cancellazione di una casa produttrice, che comporterebbe la cancellazione di tutti i film da essa prodotti).
- **Modifiche:** per le operazioni di aggiornamento invece è stato scelto l'utilizzo di `ON UPDATE CASCADE`, che non comporta effetti collaterali distruttivi.
- **Vincoli generici:** i rimanenti vincoli di integrità *inter-relazionali* sono stati invece implementati tramite *trigger* (Sezione 5.3).

```
CREATE TABLE AziendaProduttrice (  
    Nome VARCHAR(255) PRIMARY KEY,  
    Recapito VARCHAR(255) UNIQUE NOT NULL,  
    NumeroDiFilmProdotti INT NOT NULL  
);  
  
CREATE TABLE Genere (  
    Nome VARCHAR(100) PRIMARY KEY  
);  
  
CREATE TABLE ClienteRegistrato (  
    Email VARCHAR(255) PRIMARY KEY,  
    Username VARCHAR(100) UNIQUE NOT NULL,  
    Password VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE Persona (  
    Nome VARCHAR(100),  
    Cognome VARCHAR(100),  
    DataDiNascita DATE,  
  
    PRIMARY KEY (Nome, Cognome, DataDiNascita)  
);  
  
CREATE TABLE Attore (  
    Nome VARCHAR(100),  
    Cognome VARCHAR(100),  
    DataDiNascita DATE,  
  
    PRIMARY KEY (Nome, Cognome, DataDiNascita),  
    FOREIGN KEY (Nome, Cognome, DataDiNascita)  
        REFERENCES Persona(Nome, Cognome, DataDiNascita)  
    ON UPDATE CASCADE
```

```

        ON DELETE NO ACTION
    );

CREATE TABLE Regista (
    Nome VARCHAR(100),
    Cognome VARCHAR(100),
    DataDiNascita DATE,

    PRIMARY KEY (Nome, Cognome, DataDiNascita),
    FOREIGN KEY (Nome, Cognome, DataDiNascita)
        REFERENCES Persona(Nome, Cognome, DataDiNascita)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

CREATE TABLE Film (
    Titolo VARCHAR(255),
    AnnoDiProduzione INT,
    Durata INT NOT NULL CHECK (Durata > 0),
    Trama TEXT NOT NULL,
    AziendaProduttrice VARCHAR(255) NOT NULL,

    PRIMARY KEY (Titolo, AnnoDiProduzione),
    FOREIGN KEY (AziendaProduttrice)
        REFERENCES AziendaProduttrice(Nome)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

CREATE TABLE RegistaDelFilm (
    TitoloFilm VARCHAR(255),
    AnnoDiProduzioneFilm INT,
    NomeRegista VARCHAR(100),
    CognomeRegista VARCHAR(100),
    DataDiNascitaRegista DATE,

    PRIMARY KEY (TitoloFilm, AnnoDiProduzioneFilm, NomeRegista, CognomeRegista,
DataDiNascitaRegista),
    FOREIGN KEY (TitoloFilm, AnnoDiProduzioneFilm)
        REFERENCES Film(Titolo, AnnoDiProduzione)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    FOREIGN KEY (NomeRegista, CognomeRegista, DataDiNascitaRegista)
        REFERENCES Regista(Nome, Cognome, DataDiNascita)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

CREATE TABLE GenereDelFilm (
    TitoloFilm VARCHAR(255),
    AnnoDiProduzioneFilm INT,
    NomeGenere VARCHAR(100),

    PRIMARY KEY (TitoloFilm, AnnoDiProduzioneFilm, NomeGenere),
    FOREIGN KEY (TitoloFilm, AnnoDiProduzioneFilm)

```



```

        REFERENCES Film(Titolo, AnnoDiProduzione)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    FOREIGN KEY (NomeGenere)
        REFERENCES Genere(Nome)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

CREATE TABLE CopiaFisicaDiFilm (
    Numero INT,
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,

    PRIMARY KEY (Numero, TitoloFilm, AnnoFilm),
    FOREIGN KEY (TitoloFilm, AnnoFilm)
        REFERENCES Film(Titolo, AnnoDiProduzione)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

CREATE TABLE Recitazione (
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,
    NomeAttore VARCHAR(100),
    CognomeAttore VARCHAR(100),
    DataNascitaAttore DATE,

    PRIMARY KEY (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore),
    FOREIGN KEY (TitoloFilm, AnnoFilm)
        REFERENCES Film(Titolo, AnnoDiProduzione)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    FOREIGN KEY (NomeAttore, CognomeAttore, DataNascitaAttore)
        REFERENCES Attore(Nome, Cognome, DataDiNascita)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

CREATE TABLE Ruolo (
    NomeRuolo VARCHAR(100),
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,
    NomeAttore VARCHAR(100),
    CognomeAttore VARCHAR(100),
    DataNascitaAttore DATE,

    PRIMARY KEY (NomeRuolo, TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore),
    FOREIGN KEY (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore)
        REFERENCES Recitazione(TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore)
        ON UPDATE CASCADE

```

```

        ON DELETE NO ACTION
    );

CREATE TABLE FraseSignificativa (
    ID INT GENERATED BY DEFAULT AS IDENTITY,
    TitoloFilm VARCHAR(255) NOT NULL,
    AnnoFilm INT NOT NULL,
    NomeAttore VARCHAR(100) NOT NULL,
    CognomeAttore VARCHAR(100) NOT NULL,
    DataNascitaAttore DATE NOT NULL,
    Frase TEXT NOT NULL,

    PRIMARY KEY (ID),
    FOREIGN KEY (TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore)
        REFERENCES Recitazione(TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,
DataNascitaAttore)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

CREATE TABLE Noleggio (
    DataDiInizio DATE,
    NumeroCopia INT,
    TitoloFilm VARCHAR(255),
    AnnoFilm INT,
    EmailCliente VARCHAR(255) NOT NULL,
    DurataMassimaNoleggior INT NOT NULL,
    DataDiFine DATE CHECK (DataDiFine IS NULL OR DataDiFine > DataDiInizio),

    PRIMARY KEY (DataDiInizio, NumeroCopia, TitoloFilm, AnnoFilm),
    FOREIGN KEY (NumeroCopia, TitoloFilm, AnnoFilm)
        REFERENCES CopiaFisicaDiFilm(Numero, TitoloFilm, AnnoFilm)
        ON UPDATE CASCADE
        ON DELETE NO ACTION,
    FOREIGN KEY (EmailCliente)
        REFERENCES ClienteRegistrato(Email)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
);

```

Codice 1: Creazione dello Schema

5. Implementazione

Per una più semplice e veloce configurazione della base di dati è stata creata un'unica lista di comandi per la creazione delle tabelle, dei triggers, del popolamento della base di dati, e dell'esecuzione delle interrogazioni.

```
psql -d $USER \  
-c "CREATE DATABASE industria_cinematografica;"  
  
psql -d industria_cinematografica -P pager \  
-c "\i db/create.sql" \  
-c "\i db/trigger_1.sql" \  
-c "\i db/trigger_2.sql" \  
-c "\i db/trigger_3.sql" \  
-c "\i db/trigger_4.sql" \  
-c "\i db/trigger_5.sql" \  
-c "\i db/seed.sql" \  
-c "\i db/operation_1__query_1.sql" \  
-c "\i db/operation_2__query_2.sql" \  
-c "\i db/operation_3__query_3.sql" \  
-c "\i db/operation_4__query_4.sql" \  
-c "\i db/operation_6__query_5.sql" \  
-c "\i db/operation_8__query_6.sql"
```

Nel caso si volesse eseguire l'istanziatura delle operazioni di inserimento e di cancellazione con parametri di esempio, è possibile eseguire i seguenti comandi:

```
psql -d industria_cinematografica -P pager \  
-c "\i db/operation_5__insertion_1_instanced.sql" \  
-c "\i db/operation_7__insertion_2_instanced.sql" \  
-c "\i db/operation_9__deletion_1_instanced.sql"
```

Si nota che l'esecuzione ripetuta di tali comandi porta ad un fallimento, data la natura statica dei dati d'esempio inseriti o cancellati.

Al fine di porre ulteriore chiarezza, nelle sezioni successive vengono descritti questi passaggi uno a uno.

5.0.1. Docker

In alternativa al metodo di configurazione tradizionale viene anche proposto un metodo di configurazione basato su Docker, quindi più indipendente dall'ambiente di esecuzione.

Le istruzioni per la configurazione della base di dati tramite Docker non vengono riportate in questo documento, ma sono reperibili nel file *README.md* nella cartella *docker*.

5.1. Creazione della Base di Dati

Il seguente comando crea la nuova base di dati, sulla quale verranno eseguiti i seguenti comandi di creazione delle tabelle, dei trigger, del popolamento, e delle interrogazioni.

```
psql -c "CREATE DATABASE industria_cinematografica;"
```

5.2. Creazione delle Tabelle

Una volta creata la base di dati, vengono create le tabelle, con i relativi vincoli di integrità, attraverso il seguente comando:

```
psql -d industria_cinematografica -c "\i db/create.sql"
```

5.3. Implementazione dei Trigger

Vengono presentate le implementazioni dei seguenti trigger, scelti per le diverse tipologie di operazioni necessarie per mantenere i vincoli di integrità:

1. **Una persona deve essere o un attore, o un regista, o entrambi:** Trigger in inserimento su *Persona*.
2. **Ci può essere al massimo un noleggio attivo:** Trigger in modifica su *Noleggio*.
3. **Attributo derivato “Numero di Film Prodotti”:** Trigger in cancellazione su *Film*.
4. **Intervalli di noleggio non sovrapposti:** Trigger in inserimento su *Noleggio*.

Per il setup dei triggers, eseguire:

```
psql -d industria_cinematografica -c "\i db/trigger_1.sql"
psql -d industria_cinematografica -c "\i db/trigger_2.sql"
psql -d industria_cinematografica -c "\i db/trigger_3.sql"
psql -d industria_cinematografica -c "\i db/trigger_4.sql"
psql -d industria_cinematografica -c "\i db/trigger_5.sql"
```

Si nota che è stato implementato anche un quinto trigger, relativo all’aggiornamento in inserimento dell’attributo derivato “Numero di Film Prodotti”. Non viene riportato per motivi di brevità.

5.3.1. Trigger 1: Una persona deve essere o un attore, o un regista, o entrambi

```
CREATE OR REPLACE FUNCTION ControllaSpecializzazione() RETURNS trigger AS $$
BEGIN
    IF NOT EXISTS (
        SELECT *
        FROM Attore
        WHERE Nome = NEW.Nome AND Cognome = NEW.Cognome AND DataDiNascita =
NEW.DataDiNascita
    ) AND NOT EXISTS (
        SELECT *
        FROM Regista
        WHERE Nome = NEW.Nome AND Cognome = NEW.Cognome AND DataDiNascita =
NEW.DataDiNascita
    ) THEN
        RAISE EXCEPTION 'La persona deve essere o un attore, o un regista, o
entrambi';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER Persona_ControllaSpecializzazioneInInserimento
```

```

AFTER INSERT ON Persona
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION ControllaSpecializzazione();

```

5.3.2. Trigger 2: Ci può essere al massimo un noleggio attivo

```

CREATE OR REPLACE FUNCTION ControllaNoleggioAttivo() RETURNS trigger AS $$
BEGIN
    IF EXISTS (
        SELECT *
        FROM Noleggio
        WHERE
            TitoloFilm = NEW.TitoloFilm AND
            AnnoFilm = NEW.AnnoFilm AND
            NumeroCopia = NEW.NumeroCopia AND
            DataDiInizio <> NEW.DataDiInizio AND
            DataDiFine IS NULL
    ) THEN
        RAISE EXCEPTION 'Ci può essere al massimo un noleggio attivo per questa copia
        fisica di film';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER Noleggio_ControllaNoleggioAttivoInModifica
AFTER UPDATE ON Noleggio
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION ControllaNoleggioAttivo();

```

5.3.3. Trigger 3: Attributo derivato “Numero di Film Prodotti”

```

CREATE OR REPLACE FUNCTION AggiornaNumeroFilmProdotti() RETURNS trigger AS $$
BEGIN
    UPDATE AziendaProduttrice
    SET NumeroDiFilmProdotti = (
        SELECT COUNT(*)
        FROM Film
        WHERE AziendaProduttrice = OLD.AziendaProduttrice
    )
    WHERE Nome = OLD.AziendaProduttrice;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER Film_AggiornaNumeroFilmProdottiInRimozione
AFTER DELETE ON Film
FOR EACH ROW
EXECUTE FUNCTION AggiornaNumeroFilmProdotti();

```

5.3.4. Trigger 4: Intervalli di noleggio non sovrapposti

```
CREATE OR REPLACE FUNCTION ControllaIntervalliNoleggio()
RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE
    CNT INTEGER;
BEGIN
    -- Seleziona i noleggi che sono in overlap
    SELECT COUNT(*) INTO CNT
    FROM Noleggio N
    WHERE N.NumeroCopia = NEW.NumeroCopia
        AND N.TitoloFilm = NEW.TitoloFilm
        AND N.AnnoFilm = NEW.AnnoFilm
        AND (NEW.DataDiFine IS NULL OR N.DataDiInizio <= NEW.DataDiFine)
        AND (N.DataDiFine IS NULL OR NEW.DataDiInizio <= N.DataDiFine);

    IF CNT > 0
    THEN
        RAISE EXCEPTION 'Il Noleggio è invalido.';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_ControllaIntervalliNoleggio
BEFORE INSERT ON Noleggio
FOR EACH ROW
EXECUTE FUNCTION ControllaIntervalliNoleggio();
```

5.4. Popolamento della Base di Dati

Il popolamento della base di dati è stato effettuato attraverso uno script *Python*. Per la generazione di un dataset realistico si è utilizzata la libreria *faker*, capace di generare dati verosimili per vari domini, come nomi di aziende, di persone, ecc. L'obiettivo era quello di inizializzare la base di dati in modo completo, per far sì che le operazioni di Sezione 5.5 restituissero risultati significativi e corretti, rispettando i vincoli di integrità e quindi evitare errori da parte dei trigger.

L'utilizzo di questi strumenti ha permesso di popolare la base di dati, anche se con volumi ridotti rispetto a quelli proposti in Sezione 3.1.1.

È importante notare che l'ordine degli inserimenti deve rispettare i vincoli di integrità definiti.

In particolare, sono state evidenziate alcune dipendenze cicliche tra relazioni, causate dalla partecipazione obbligatoria delle relazioni coinvolte. Per queste dipendenze cicliche, l'inserimento deve avvenire all'interno in una o più transazioni con il controllo dei vincoli posticipato (DEFERRED) al COMMIT.

Segue la lista delle dipendenze problematiche:

- *Persona* ↔ *Attore/Regista*: una *Persona* deve essere o un *Attore* o un *Regista* o entrambi; un *Attore/Regista* deve essere una *Persona*
- *Azienda* ↔ *Film*: un'*Azienda* deve avere prodotto almeno un *Film*; un *Film* deve essere prodotto da un'*Azienda*

- *Film* ↔ *Genere*: un *Film* deve avere un *Genere*; un *Genere* deve essere associato ad un *Film*
- *Film* ↔ *Regista*: un *Film* deve avere un *Regista* che lo dirige; un *Regista* deve dirigere un *Film*
- *Attore* ↔ *Recitazione* ↔ *Ruolo*: un *Attore* deve partecipare ad almeno una *Recitazione*; la *Recitazione* si riferisce a un *Attore* e richiede almeno un *Ruolo*.

Di conseguenza, tutti i dati relativi alle tabelle *Azienda*, *Film*, *Generi*, *Registi* (e *Persone*), *RegistaDelFilm* e *GenereDelFilm* devono essere inseriti in un'unica transazione.

Lo stesso vale per *Recitazioni*, *Attori* (e *Persone*), e *Ruoli*.

Per le rimanenti relazioni non è necessario posticipare il controllo dei vincoli, ma i seguenti ordini di inserimento devono essere rispettati:

- *Noleggio*: l'inserimento deve avvenire successivamente all'inserimento del relativo *Cliente Registrato* e della relativa *Copia fisica di film* (tra *Cliente Registrato* e *Copia fisica di film* non è necessario un ordinamento);
- *Frase Significativa*: possono essere inserite in qualunque momento successivo all'inserimento della relativa *Recitazione*.

In definitiva, lo script di inserimento segue il seguente ordine di inserimenti:

1. Crea una transazione in cui inserisce *Aziende*, *Film*, *Generi*, *Registi* (e le associate *Persone*), *RegistaDelFilm* e *GenereDelFilm*, con vincoli DEFERRED.
2. Crea una seconda transazione in cui inserisce *Recitazioni*, *Attori* (e le associate *Persone*), e *Ruoli*, con vincoli DEFERRED.
3. Inserisce, in questo ordine, *ClienteRegistrato*, *CopiaFisicaDiFilm* e *Noleggio*.
4. Inserisce *FraseSignificativa*.

Allo scopo di rendere la configurazione del database più semplice, lo script Python non inizializza la base di dati direttamente, ma crea un file `seed.sql` con le istruzioni di inserimento. In questo modo non è necessario installare le dipendenze richieste dallo script.

Lo script viene fornito per completezza, tuttavia, è consigliato usare il file `seed.sql`, come suggerito nelle istruzioni precedenti.

5.5. Interrogazioni ed Operazioni

Vengono proposte in seguito le interrogazioni ed operazioni in riferimento a quanto definito in Sezione 1.3.

Notiamo che, per quanto riguarda le operazioni di inserimento e di cancellazione, tutti i parametri prefissati con \$ sono da considerarsi come variabili, e dunque da sostituire con i valori di input desiderati. Per ogni operazione utilizzando questi parametri, è possibile trovare un esempio con valori di input specifici nei file terminanti con `_instanced` (per esempio, `operation_5__insertion_1__instanced.sql`).

5.5.1. Operazione 1

```
-- Troviamo per ogni film il numero di attori che vi recitano. Notiamo che ogni
istanza di recitazione corrisponde ad un ed un solo un attore. Non serve quindi
effettuare join con attore
CREATE OR REPLACE VIEW NumeroAttoriPerFilm AS
SELECT Film.Titolo AS TitoloFilm,
       Film.AnnoDiProduzione AS AnnoFilm,
       COUNT(Recitazione.NomeAttore) AS NumeroAttori
```

```

FROM Film LEFT JOIN Recitazione ON
    Film.Titolo = Recitazione.TitoloFilm AND
    Film.AnnoDiProduzione = Recitazione.AnnoFilm
GROUP BY Film.Titolo, Film.AnnoDiProduzione;

-- Utilizziamo la vista precedente per ottenere il numero medio di attori per
-- genere
SELECT GenereDelFilm.NomeGenere, AVG(NumeroAttori)
FROM GenereDelFilm
JOIN NumeroAttoriPerFilm ON
    GenereDelFilm.TitoloFilm = NumeroAttoriPerFilm.TitoloFilm AND
    GenereDelFilm.AnnoDiProduzioneFilm = NumeroAttoriPerFilm.AnnoFilm
GROUP BY GenereDelFilm.NomeGenere;

```

5.5.2. Operazione 2

```

SELECT C1.Email, C2.Email
FROM ClienteRegistrato C1, ClienteRegistrato C2
WHERE
    NOT EXISTS ( -- non esiste un Film che ha visto C1 ma non C2
        SELECT *
        FROM Noleggio N1
        WHERE N1.EmailCliente = C1.Email AND
            NOT EXISTS (
                SELECT *
                FROM Noleggio N2
                WHERE N2.EmailCliente = C2.Email AND
                    N1.TitoloFilm = N2.TitoloFilm AND
                    N1.AnnoFilm = N2.AnnoFilm
            )
    )
    AND
    NOT EXISTS ( -- non esiste un Film che ha visto C2 ma non C1
        SELECT *
        FROM Noleggio N1
        WHERE N1.EmailCliente = C2.Email AND
            NOT EXISTS (
                SELECT *
                FROM Noleggio N2
                WHERE N2.EmailCliente = C1.Email AND
                    N1.TitoloFilm = N2.TitoloFilm AND
                    N1.AnnoFilm = N2.AnnoFilm
            )
    )
    AND C1.Email < C2.Email;

```

5.5.3. Operazione 3

```

CREATE OR REPLACE VIEW NumeroFilmPerRegista(NomeRegista, CognomeRegista,
DataDiNascitaRegista, NumeroFilm) AS
SELECT Regista.Nome, Regista.Cognome, Regista.DataDiNascita, COUNT(*)
FROM Regista
JOIN RegistaDelFilm ON RegistaDelFilm.NomeRegista = Regista.Nome AND

```



```

        RegistaDelFilm.CognomeRegista = Regista.Cognome AND
        RegistaDelFilm.DataDiNascitaRegista = Regista.DataDiNascita
    GROUP BY Regista.Nome, Regista.Cognome, Regista.DataDiNascita;

    SELECT NomeRegista, CognomeRegista, DataDiNascitaRegista, NumeroFilm
    FROM NumeroFilmPerRegista
    WHERE
        NumeroFilm >= ALL (
            SELECT NumeroFilm
            FROM NumeroFilmPerRegista
        );

```

5.5.4. Operazione 4

```

CREATE OR REPLACE VIEW Attore_AziendeProd AS
    SELECT NomeAttore, CognomeAttore, DataNascitaAttore, AziendaProduttrice
    FROM   Recitazione JOIN Film
        ON  Recitazione.TitoloFilm = Film.Titolo
        AND Recitazione.AnnoFilm   = Film.AnnoDiProduzione;

    SELECT DISTINCT A1.NomeAttore, A1.CognomeAttore, A1.DataNascitaAttore
    FROM   Attore_AziendeProd A1
    WHERE  NOT EXISTS (
        SELECT *
        FROM   Attore_AziendeProd A2
        WHERE  A1.NomeAttore      = A2.NomeAttore
        AND    A1.CognomeAttore   = A2.CognomeAttore
        AND    A1.DataNascitaAttore = A2.DataNascitaAttore
        AND    A1.AziendaProduttrice <> A2.AziendaProduttrice
    )

```

5.5.5. Operazione 5

L'operazione 5, è di inserimento. Notiamo che l'aggiornamento dell'attributo derivato *Numero di Film Prodotti* in *Azienda Produttrice* è gestito da un trigger, quindi non è necessario occuparsene nell'operazione di inserimento.

```

BEGIN TRANSACTION;

INSERT INTO Film VALUES (
    $TitoloFilm,
    $AnnoFilm,
    $DurataFilm,
    $TramaFilm,
    $AziendaProduttrice
);

INSERT INTO RegistaDelFilm VALUES (
    $TitoloFilm,
    $AnnoFilm,
    $NomeRegista,
    $CognomeRegista,
    $DataDiNascitaRegista

```

```
);  
  
COMMIT;
```

5.5.6. Operazione 6

L'operazione 6, risulta estremamente semplice in quanto usa l'attributo derivato, dato che la ridondanza è stata mantenuta.

Si nota che nel prototipo della base di dati è stato solo implementato il trigger per la rimozione di film. Segue che i valori ritornati dall'esecuzione della query nel prototipo non risulteranno aggiornati.

```
SELECT Nome, NumeroDiFilmProdotti  
FROM AziendaProduttrice;
```

5.5.7. Operazione 7

```
BEGIN TRANSACTION;  
  
INSERT INTO Recitazione VALUES (  
    $TitoloFilm,  
    $AnnoFilm,  
    $NomeAttore,  
    $CognomeAttore,  
    $DataDiNascitaAttore  
);  
  
INSERT INTO Ruolo VALUES (  
    $NomeRuolo,  
    $TitoloFilm,  
    $AnnoFilm,  
    $NomeAttore,  
    $CognomeAttore,  
    $DataDiNascitaAttore  
);  
  
INSERT INTO FraseSignificativa(TitoloFilm, AnnoFilm, NomeAttore, CognomeAttore,  
DataNascitaAttore)  
VALUES (  
    $TitoloFilm,  
    $AnnoFilm,  
    $NomeAttore,  
    $CognomeAttore,  
    $DataDiNascitaAttore,  
    $Frase  
);  
  
COMMIT;
```

5.5.8. Operazione 8

Avendo eliminato l'attributo derivato *numero di film recitati*, il numero di film deve essere calcolato.

```
SELECT NomeAttore, CognomeAttore, DataNascitaAttore, COUNT(*) AS NumeroFilm
FROM Recitazione
GROUP BY NomeAttore, CognomeAttore, DataNascitaAttore;
```

5.5.9. Operazione 9

L'operazione 9 è di cancellazione. Notiamo tuttavia che, a causa dell'utilizzo di **ON DELETE NO ACTION** per la maggior parte delle chiavi esterne, è necessario cancellare anche tutte le istanze correlate al film da cancellare. A causa delle dipendenze cicliche tra le relazioni, è necessario eseguire questa operazione in una unica transazione.

Questa operazione di rimozione è implementata per puro scopo dimostrativo, in quanto nella vera base di dati, la cancellazione di entità dovrebbe essere solo un caso eccezionale, e non una pratica comune.

```
BEGIN TRANSACTION;

DELETE
FROM Ruolo
WHERE
    TitoloFilm = $TitoloFilm AND
    AnnoFilm = $AnnoFilm;

DELETE
FROM FraseSignificativa
WHERE
    TitoloFilm = $TitoloFilm AND
    AnnoFilm = $AnnoFilm;

DELETE
FROM Recitazione
WHERE
    TitoloFilm = $TitoloFilm AND
    AnnoFilm = $AnnoFilm;

DELETE
FROM GenereDelFilm
WHERE
    TitoloFilm = $TitoloFilm AND
    AnnoDiProduzioneFilm = $AnnoFilm;

DELETE
FROM RegistaDelFilm
WHERE
    TitoloFilm = $TitoloFilm AND
    AnnoDiProduzioneFilm = $AnnoFilm;

DELETE
FROM Noleggio
WHERE
```

```

        TitoloFilm = $TitoloFilm AND
        AnnoFilm = $AnnoFilm;

DELETE
FROM CopiaFisicaDiFilm
WHERE
    TitoloFilm = $TitoloFilm AND
    AnnoFilm = $AnnoFilm;

DELETE
FROM Film
WHERE
    Titolo = $TitoloFilm AND
    AnnoDiProduzione = $AnnoFilm;

DELETE
FROM Genere
WHERE
    (SELECT COUNT(*) FROM GenereDelFilm WHERE NomeGenere = Genere.Nome) = 0;

DELETE
FROM Attore
WHERE
    (SELECT COUNT(*) FROM Recitazione WHERE NomeAttore = Attore.Nome AND
    CognomeAttore = Attore.Cognome AND DataNascitaAttore = Attore.DataDiNascita) = 0;

DELETE
FROM Regista
WHERE
    (SELECT COUNT(*) FROM RegistaDelFilm WHERE NomeRegista = Regista.Nome AND
    CognomeRegista = Regista.Cognome AND DataDiNascitaRegista = Regista.DataDiNascita)
    = 0;

DELETE
FROM Persona
WHERE
    (SELECT COUNT(*) FROM Attore WHERE Nome = Persona.Nome AND Cognome =
    Persona.Cognome AND DataDiNascita = Persona.DataDiNascita) = 0
    AND
    (SELECT COUNT(*) FROM Regista WHERE Nome = Persona.Nome AND Cognome =
    Persona.Cognome AND DataDiNascita = Persona.DataDiNascita) = 0;

-- Notiamo che il numero di film prodotti è già aggiornato dal trigger
-- associato, in quanto non impostato come deferred.
DELETE
FROM AziendaProduttrice
WHERE
    AziendaProduttrice.NumeroDiFilmProdotti = 0;

COMMIT;

```

6. Conclusioni

In questo progetto è stata progettata una base di dati per un'industria cinematografica, partendo da un modello concettuale, passando per un modello logico, fino ad arrivare alla progettazione fisica, con la creazione dello schema in SQL DDL, l'implementazione dei trigger, il popolamento della base di dati e l'esecuzione di alcune interrogazioni.

Si è capito come progettare ed implementare una solida base di dati, in particolar modo, come strutturare il processo nelle sue varie fasi.

Le competenze acquisite in questo progetto sono molteplici, e resteranno sicuramente utili per la progettazione di basi di dati in futuro, sia a livello accademico che professionale.